Homework 4

In this homework, we will cover panorama stitching.

This homework was adapted from Stanford CS131.

Setup

In [64]:

```
'''
import os

if not os.path.exists("Emory-Computer-Vision-Course"):
    # Clone the repository if it doesn't already exist
    !git clone https://github.com/YoungjoongEmory/Emory-Computer-Vision-Course.git

%cd Emory-Computer-Vision-Course/Assignments/HW4/
'''
```

Out[64]:

```
'\nimport os\n\nif not os.path.exists("Emory-Computer-Vision-Course"):\n    # Clone the repository if it doesn\'t already exist\n    !git clone https://
github.com/YoungjoongEmory/Emory-Computer-Vision-Course.git\n\n%cd Emory-Computer-Vision-Course/Assignments/HW4/\n'
```

In [65]:

```
# Install the necessary dependencies
# (restart your runtime session if prompted to, and then re-run this cell)
!pip install -r requirements.txt
```

```
Requirement already satisfied: numpy in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from -r requirements.txt (line 1)) (2.5.2)
Requirement already satisfied: scikit-image in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from -r requirements.txt (line 2)) (
0.26.0)
Requirement already satisfied: scipy in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from -r requirements.txt (line 3)) (1.18.1)
Requirement already satisfied: matplotlib in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from -r requirements.txt (line 4)) (3.
11.0)
Requirement already satisfied: opencv-python in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from -r requirements.txt (line 5))
(4.14.0)
Requirement already satisfied: networkx>=3.0 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from scikit-image->-r requirements.
txt (line 2)) (3.6.1)
Requirement already satisfied: pillow>=10.1 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from scikit-image->-r requirements.t
xt (line 2)) (12.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from scikit-image->-r requ
irements.txt (line 2)) (2.37.4)
Requirement already satisfied: tifffile>=2022.8.12 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from scikit-image->-r require
ments.txt (line 2)) (2026.9.9)
Requirement already satisfied: packaging>=21 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from scikit-image->-r requirements.
txt (line 2)) (26.3)
Requirement already satisfied: lazy-loader>=0.4 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from scikit-image->-r requiremen
ts.txt (line 2)) (0.5)
Requirement already satisfied: contourpy>=1.0.1 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from matplotlib->-r requirement
s.txt (line 4)) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt
(line 4)) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from matplotlib->-r requirement
s.txt (line 4)) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from matplotlib->-r requirement
s.txt (line 4)) (1.5.0)
Requirement already satisfied: pyparsing>=3 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt
(line 4)) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from matplotlib->-r requirem
ents.txt (line 4)) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /Users/nataliehu/anaconda3/envs/emory_cv/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib->-
r requirements.txt (line 4)) (1.17.0)
```

In [66]:

```
from __future__ import print_function

import numpy as np
from skimage import filters
from skimage.feature import corner_peaks
from skimage.io import imread
from scipy.spatial.distance import cdist
from scipy.ndimage import convolve
from scipy import ndimage
import matplotlib.pyplot as plt
import cv2 as cv

from utils import pad, unpad, get_output_space, warp_image, plot_matches, describe_keypoints

%matplotlib inline
plt.rcParams['figure.figsize'] = (12.0, 9.0)
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'
```

Panorama Stitching

Panorama stitching is an early success of computer vision. Matthew Brown and David G. Lowe published a famous panoramic image stitching paper in 2007. Since then, automatic panorama stitching technology has been widely adopted in many applications such as Google Street View, panorama photos on smartphones, and stitching software such as Photosynth and AutoStitch.

We will detect and match keypoints from multiple images to build a single panoramic image. This will involve several tasks:

1. Compare two sets of descriptors coming from two different images and find matching keypoints.
2. Given matching keypoints, use least-squares to find the affine transformation matrix that maps points in one image to another.
3. Use RANSAC to give a more robust estimate of the affine transformation matrix.
   Given the transformation matrix, use it to transform the second image and overlay it on the first image, forming a panorama.
4. Blend panorama images together to remove blurry regions of overlapping images.

1. Describing and Matching Keypoints (0 points)

We will use Harris corner detection to detect keypoints in two images. Given these two sets of keypoints, we will then determine which pairs of keypoints come from the same 3D points projected onto the two images. We do so by first converting the region around each keypoint into a descriptor. Then, we find good matches in the two sets of descriptors based on Euclidean distance.

Use the following definition of cornerness response:

$$\theta = \det(M) - \alpha(\operatorname{trace}(M))^2 = (w * I_x^2)(w * I_y^2) - (w * (I_x I_y))^2 - \alpha(w * (I_x^2) + w * (I_y^2))^2$$

where (*) denotes convolution and (w) is the window function.

In [67]:

```
def harris_corners(img, window_size=3, k=0.04):
    """
    Compute Harris corner response map. Follow the math equation
    R=Det(M)-k(Trace(M)^2).
    Hint:
        You may use the function ndimage.convolve,
        which is already imported above. If you use convolve(), remember to
        specify zero-padding to match our equations, for example:

            out_image = ndimage.convolve(in_image, kernel, mode='constant', cval=0)

    Args:
        img: Grayscale image of shape (H, W)
        window_size: size of the window function
        k: sensitivity parameter

    Returns:
        response: Harris response image of shape (H, W)
    """

    H, W = img.shape
    window = np.ones((window_size, window_size)) # window function

    response = np.zeros((H, W))

    # 1. Compute x and y derivatives (I_x, I_y) of an image
    dx = filters.sobel_v(img)
    dy = filters.sobel_h(img)

    ### YOUR CODE HERE

    # Compute products of derivatives using dx and dy.
    Ixx = dx * dx
    Iyy = dy * dy
    Ixy = dx * dy

    # Sum of products within the window (use zero padding)
    Sxx = convolve(Ixx, window, mode= 'constant', cval= 0)
    Syy = convolve(Iyy, window, mode= 'constant', cval= 0)
    Sxy = convolve(Ixy, window, mode= 'constant', cval= 0)

    # Harris response: R = det(M) - k * (trace(M))^2
    detM = Sxx*Syy - Sxy**2
    traceM = Sxx + Syy
    response = detM - k * (traceM)**2


    ### END YOUR CODE

    return response

def simple_descriptor(patch):
    """
    Describe the patch by normalizing the image values into a standard
    normal distribution (having mean of 0 and standard deviation of 1)
    and then flattening into a 1D array.

    The normalization will make the descriptor more robust to change
    in lighting condition.

    Hint:
        In this case of normalization, if a denominator is zero, divide by 1 instead.

    Args:
        patch: grayscale image patch of shape (H, W)

    Returns:
        feature: 1D array of shape (H * W)
    """

    feature = []
    patch = patch.astype(np.float32)
    mu = np.mean(patch)
    sigma = np.std(patch)

    ### YOUR CODE HERE

    # Normalize patch to zero mean and unit variance; handle zero std

    if sigma == 0:
        sigma = 1
    norm = (patch - mu)/sigma

    ### END YOUR CODE

    feature = norm.flatten()

    return feature

def match_descriptors(desc1, desc2, threshold=0.5):
    """
    Match the feature descriptors by finding distances between them. A match is formed
    when the distance to the closest vector is much smaller than the distance to the
    second-closest, that is, the ratio of the distances should be STRICTLY SMALLER
    than the threshold (NOT equal to). Return the matches as pairs of vector indices.

    Hint:
        The Numpy functions np.sort, np.argmin, np.asarray might be useful

        The Scipy function cdist calculates Euclidean distance between all
        pairs of inputs
    Args:
        desc1: an array of shape (M, P) holding descriptors of size P about M keypoints
        desc2: an array of shape (N, P) holding descriptors of size P about N keypoints

    Returns:
        matches: an array of shape (Q, 2) where each row holds the indices of one pair
        of matching descriptors
    """

    matches = []

    M = desc1.shape[0]

    # dists is (M, N); each row corresponds to one descriptor from image 1
    # and distances to all descriptors in image 2
    dists = cdist(desc1, desc2)

    # For each descriptor in desc1, apply Lowe's ratio test against desc2

    # dists is (M, N); for each row, find indices of two smallest distances
    idx_sorted = np.argsort(dists, axis=1)
    nearest = idx_sorted[:, 0]
    second_nearest = idx_sorted[:, 1]

    # Gather corresponding smallest and second smallest distances
    d1 = dists[np.arange(M), nearest]
    d2 = dists[np.arange(M), second_nearest]

    # Ratio test
    valid = d1 / (d2 + 1e-12) < threshold
    for i in np.where(valid)[0]:
        matches.append([i, nearest[i]])

    matches = np.asarray(matches, dtype=int) if len(matches) > 0 else np.zeros((0, 2), dtype=int)

    return matches
```

Run the following cell to detect and match the keypoints in two images.

In [68]:

```
img1 = imread('uttower1.jpg', as_gray=True)
img2 = imread('uttower2.jpg', as_gray=True)

np.random.seed(131)

# Detect keypoints in two images
keypoints1 = corner_peaks(harris_corners(img1, window_size=3),
                          threshold_rel=0.05,
                          exclude_border=8)
keypoints2 = corner_peaks(harris_corners(img2, window_size=3),
                          threshold_rel=0.05,
                          exclude_border=8)

# Extract features from the corners
desc1 = describe_keypoints(img1, keypoints1,
                           desc_func=simple_descriptor,
                           patch_size=5)
desc2 = describe_keypoints(img2, keypoints2,
                           desc_func=simple_descriptor,
                           patch_size=5)

# Match descriptors in image1 to those in image2
matches = match_descriptors(desc1, desc2, 0.7)
```

2. Transformation Estimation (20 points)

Now, we will use these matched keypoints to find a **transformation matrix** that maps points in the second image to the corresponding coordinates in the first image. In other words, if the point $p_1 = [y_1, x_1]$ in image 1 matches with $p_2 = [y_2, x_2]$ in image 2, we need to find an affine transformation matrix H such that

$$\tilde{p_2}H = \tilde{p_1},$$

where $\tilde{p_1}$ and $\tilde{p_2}$ are homogenous coordinates of p_1 and p_2.

Note that it may be impossible to find the transformation H that maps every point in image 2 exactly to the corresponding point in image 1. However, **we can estimate the transformation matrix with the least squares method.** Given N matched keypoint pairs, let X_1 and X_2 be $N \times 3$ matrices whose rows are homogenous coordinates of corresponding keypoints in image 1 and image 2 respectively. Then, we can estimate H by solving the least squares problem,

$$X_2 H = X_1$$

Implement `fit_affine_matrix` below.

Hint: read the documentation for `np.linalg.lstsq`

In [69]:

```
def fit_affine_matrix(p1, p2, to_pad=True):
    """
    Fit affine matrix such that p2 * H = p1. First, pad the descriptor vectors
    with a 1 using pad() to convert to homogeneous coordinates, then return
    the least squares fit affine matrix in homogeneous coordinates.

    Hint:
        You can use np.linalg.lstsq function to solve the problem.

        Explicitly specify np.linalg.lstsq's new default parameter rcond=None
        to suppress deprecation warnings.

    Args:
        p1: an array of shape (M, P) holding descriptors of size P about M keypoints
        p2: an array of shape (M, P) holding descriptors of size P about M keypoints

    Return:
        H: a matrix of shape (P+1, P+1) that transforms p2 to p1 in homogeneous
        coordinates
    """

    assert (p1.shape[0] == p2.shape[0]),\
        'Different number of points in p1 and p2'

    if to_pad:
        p1 = pad(p1)
        p2 = pad(p2)

    ### YOUR CODE HERE
    H = np.linalg.lstsq(p2, p1, rcond= None)[0]

    ### END YOUR CODE

    # Sometimes numerical issues cause least-squares to produce the last
    # column which is not exactly [0, 0, 1]
    H[:,2] = np.array([0, 0, 1])
    return H
```

In [70]:

```
# Sanity check for fit_affine_matrix

# Test inputs
a = np.array([[0.5, 0.1], [0.4, 0.2], [0.8, 0.2]])
b = np.array([[0.3, -0.2], [-0.4, -0.9], [0.1, 0.1]])

H = fit_affine_matrix(b, a)

# Target output
sol = np.array(
    [[1.25, 2.5, 0.0],
     [-5.75, -4.5, 0.0],
     [0.25, -1.0, 1.0]]
)

error = np.sum((H - sol) ** 2)

if error < 1e-20:
    print('Implementation correct!')
else:
    print('There is something wrong.')
```

```
Implementation correct!
```

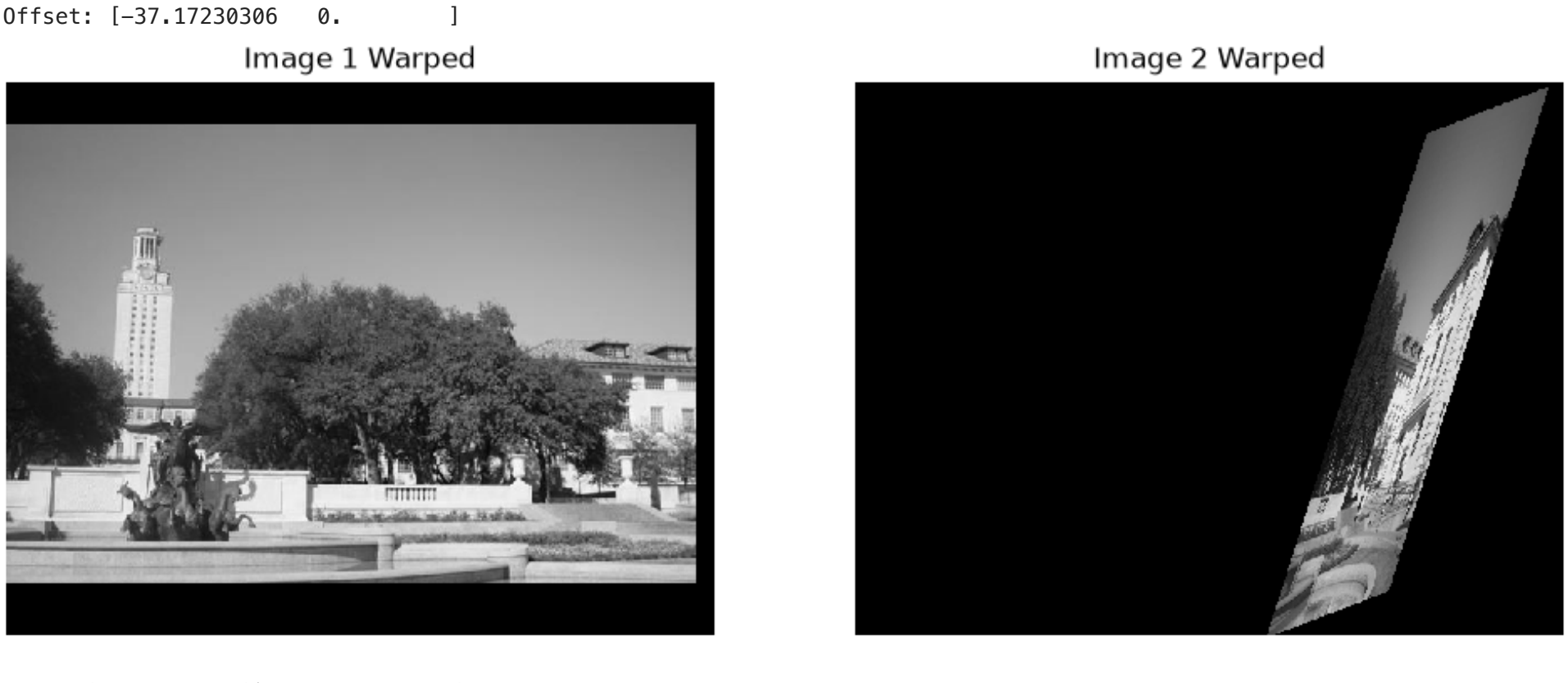
After checking that your `fit_affine_matrix` function is running correctly, run the following code to apply it to images.

Images will be warped and image 2 will be mapped to image 1.

In [71]:

```
p1 = keypoints1[matches[:,0]]
p2 = keypoints2[matches[:,1]]

# Find affine transformation matrix H that maps p2 to p1
H = fit_affine_matrix(p1, p2)

output_shape, offset = get_output_space(img1, [img2], [H])
print("Output shape:", output_shape)
print("Offset:", offset)

# Warp images into output space
img1_warped = warp_image(img1, np.eye(3), output_shape, offset)
img1_mask = (img1_warped != -1) # Mask == 1 inside the image
img1_warped[~img1_mask] = 0     # Return background values to 0

img2_warped = warp_image(img2, H, output_shape, offset)
img2_mask = (img2_warped != -1) # Mask == 1 inside the image
img2_warped[~img2_mask] = 0     # Return background values to 0

# Plot warped images
plt.subplot(1,2,1)
plt.imshow(img1_warped)
plt.title('Image 1 Warped')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(img2_warped)
plt.title('Image 2 Warped')
plt.axis('off')

plt.show()
```

```
Output shape: [493 631]
Offset: [-37.17230306   0.        ]
```

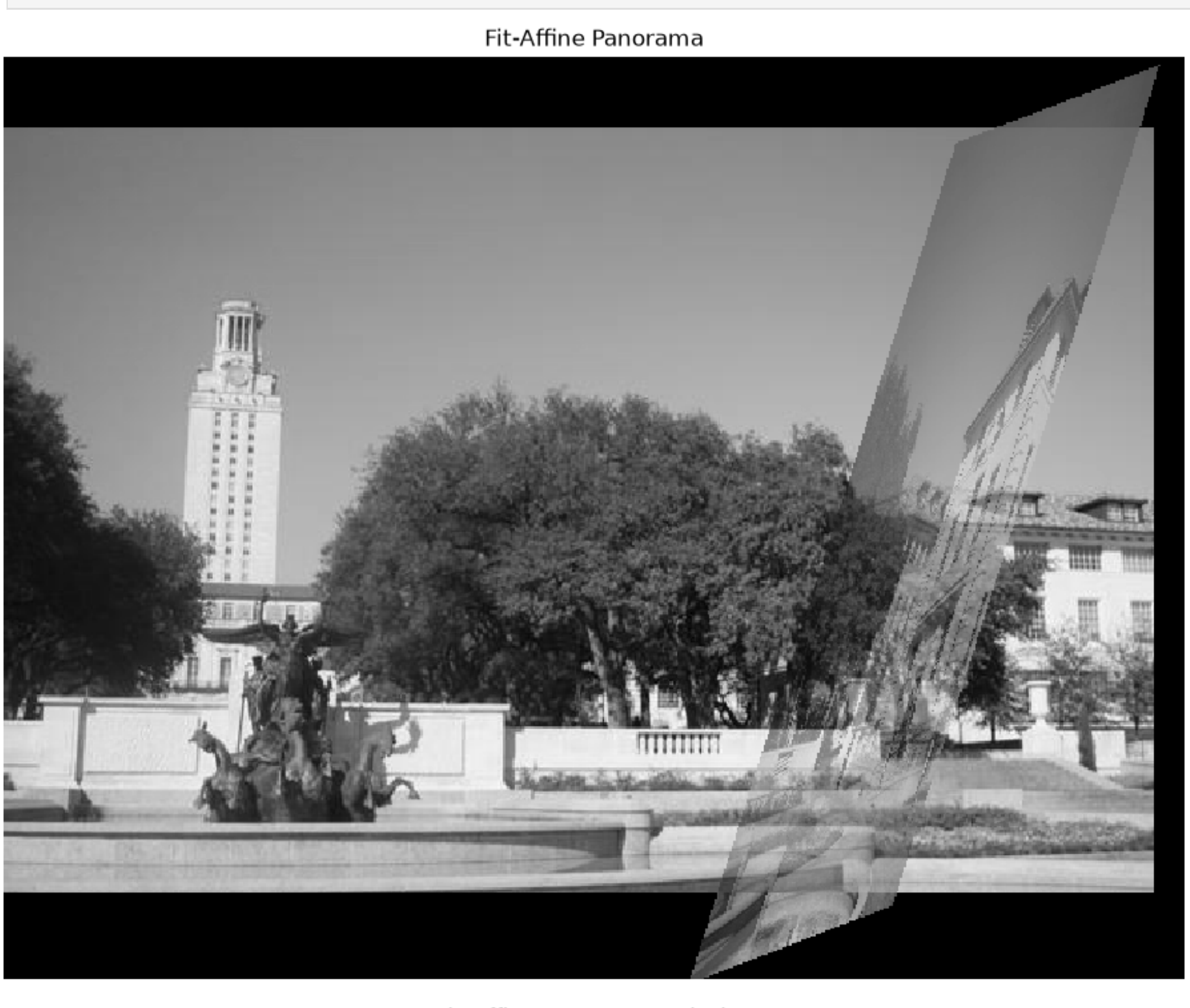


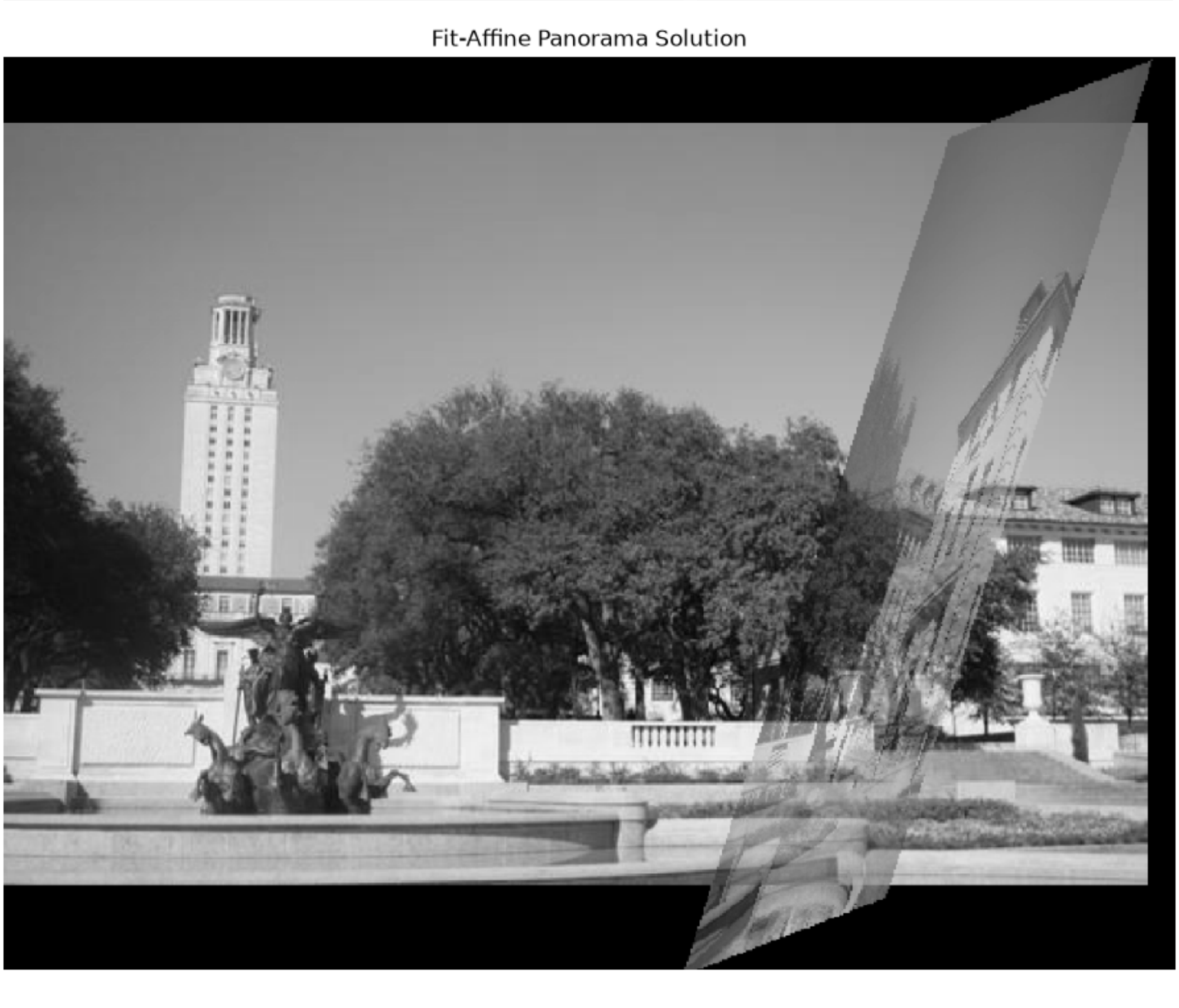
Next, the two warped images are merged to get a panorama.

Your panorama may not look good at this point, but we will later use other techniques to get a better result!

In [72]:

```
# Merge the two images
merged = img1_warped + img2_warped

# Track the overlap by adding the masks together
overlap = (img1_mask * 1.0 +  # Multiply by 1.0 for bool -> float conversion
           img2_mask)

# Normalize through division by `overlap` - but ensure the minimum is 1
normalized = merged / np.maximum(overlap, 1)

plt.imshow(normalized)
plt.axis('off')
plt.title('Fit-Affine Panorama')
plt.show()

plt.imshow(imread('solution_fit_affine_panorama.png'))
plt.axis('off')
plt.title('Fit-Affine Panorama Solution')
plt.show()
```





3. RANSAC (20 points)

Rather than directly feeding all our keypoint matches into `fit_affine_matrix`, we can use **RANSAC** ("RANdom SAmple Consensus") to select only "inliers" to use for computing the transformation matrix.

Use Euclidean distance as a measure of inliers vs. outliers.

The steps of RANSAC are:

1. Select random set of matches
2. Compute affine transformation matrix
   - You can call your `fit_affine_matrix` function for this. **Make sure to explicitly pass in** `to_pad=False`.
3. Find inliers using the given threshold
4. Repeat and keep the largest set of inliers (use >, i.e. break ties by whichever set is seen first)
5. Re-compute least-squares estimate on all of the inliers

Implement `ransac` below.

In [73]:

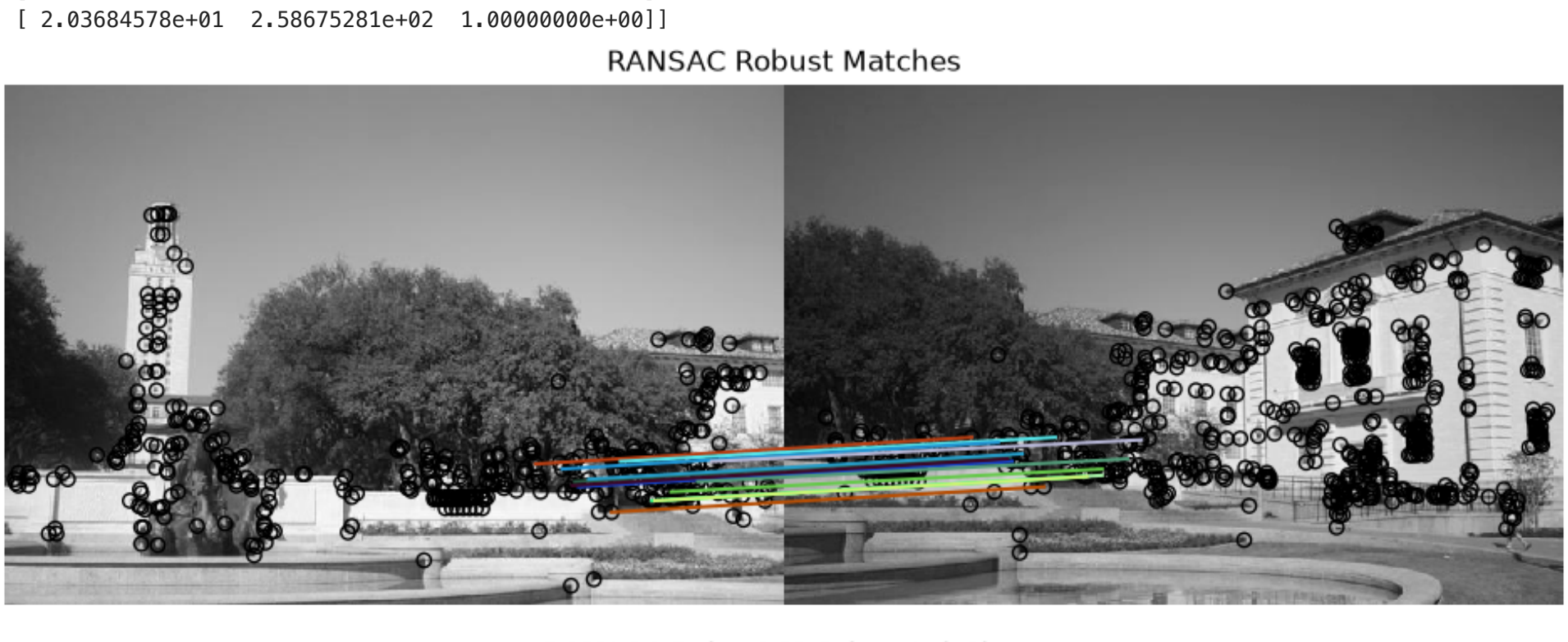
```
def ransac(keypoints1, keypoints2, matches, n_iters=200, threshold=20):
    """
    Use RANSAC to find a robust affine transformation:

        1. Select random set of matches
        2. Compute affine transformation matrix
        3. Compute inliers via Euclidean distance
        4. Keep the largest set of inliers (use >, i.e. break ties by whichever set is seen first)
        5. Re-compute least-squares estimate on all of the inliers

    Update max_inliers as a boolean array where True represents the keypoint
    at this index is an inlier, while False represents that it is not an inlier.

    Hint:
        You can use fit_affine_matrix to compute the affine transformation matrix.
        Make sure to pass in to_pad=False, since we pad the matches for you here.

        You can compute elementwise boolean operations between two numpy arrays,
        and use boolean arrays to select array elements by index:
        https://numpy.org/doc/stable/reference/arrays.indexing.html#boolean-array-indexing

    Args:
        keypoints1: M1 x 2 matrix, each row is a point
        keypoints2: M2 x 2 matrix, each row is a point
        matches: N x 2 matrix, each row represents a match
            [index of keypoint1, index of keypoint 2]
        n_iters: the number of iterations RANSAC will run
        threshold: the threshold to find inliers

    Returns:
        H: a robust estimation of affine transformation from keypoints2 to
        keypoints 1
    """

    # Copy matches array, to avoid overwriting it
    orig_matches = matches.copy()
    matches = matches.copy()

    N = matches.shape[0]
    n_samples = int(N * 0.2)

    matched1 = pad(keypoints1[matches[:,0]])
    matched2 = pad(keypoints2[matches[:,1]])

    max_inliers = np.zeros(N, dtype=bool)
    n_inliers = 0

    # RANSAC iteration start

    # Note: while there're many ways to do random sampling, we use
    # `np.random.shuffle()` followed by slicing out the first `n_samples`
    # matches here in order to align with the autograder.
    # Sample with this code:
    for i in range(n_iters):
        # 1. Select random set of matches
        np.random.shuffle(matches)
        samples = matches[:n_samples]
        sample1 = pad(keypoints1[samples[:,0]])
        sample2 = pad(keypoints2[samples[:,1]])

    ### YOUR CODE HERE

        # 2. Compute affine transformation matrix, map sample2 to sample1
        a_matrix = fit_affine_matrix(sample1, sample2, to_pad = False)

        # 3. Compute inliers via Euclidean distance
        predicted = matched2 @ a_matrix
        dist = np.linalg.norm(matched1 - predicted, axis = 1)
        inliers = dist < threshold



        # 4. Keep the largest set of inliers
        if np.sum(inliers) > np.sum(max_inliers):
            max_inliers = inliers



    # 5. Re-compute least-squares estimate on all of the inliers
    if max_inliers.any():
        H = fit_affine_matrix(matched1[max_inliers], matched2[max_inliers], to_pad=False)

    else: # Fallback: use identity if no inliers found
        H = np.identity(3)
    ### END YOUR CODE

    return H, orig_matches[max_inliers]
```

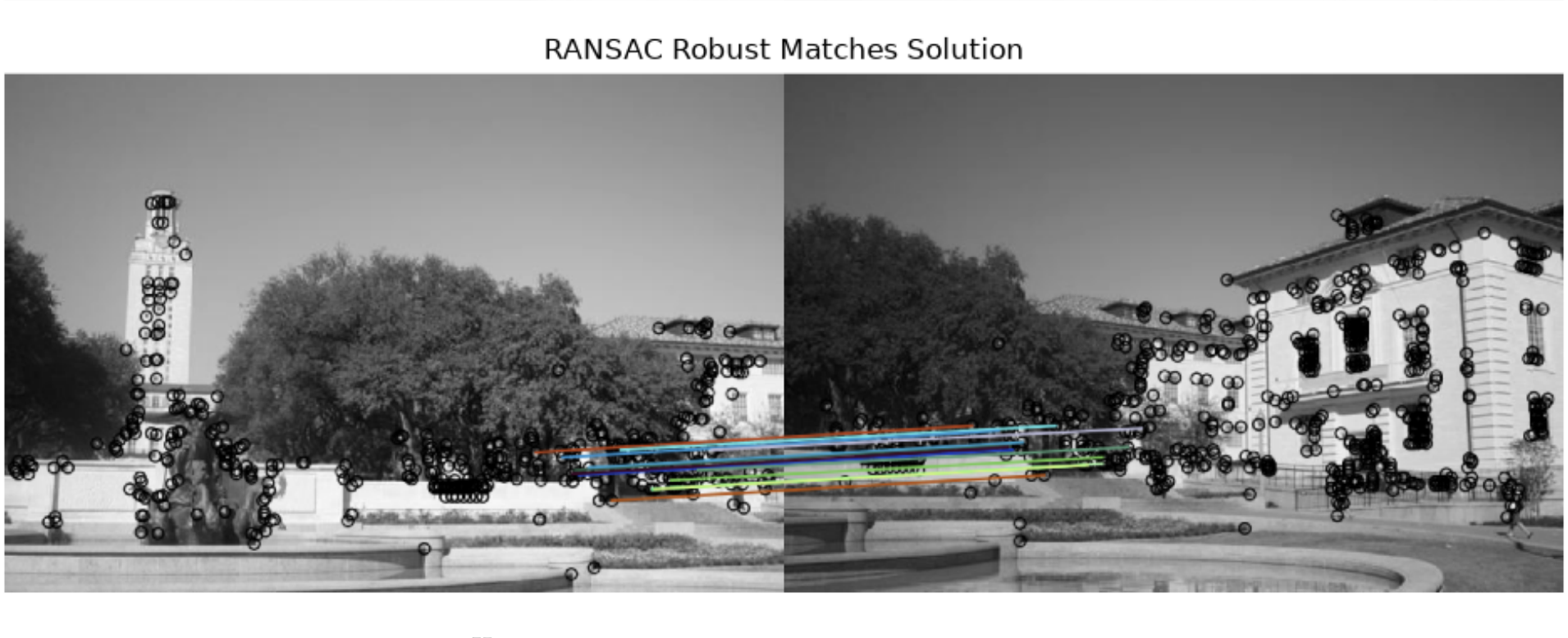
Now, run through the following cells to get a panorama. You'll be able to see the difference from the result we got before without RANSAC.

In [74]:

```
# Set seed to compare output against solution image
np.random.seed(131)

H, robust_matches = ransac(keypoints1, keypoints2, matches, threshold=1)
print("Robust matches shape = ", robust_matches.shape)
print("H = \n", H)

# Visualize robust matches
fig, ax = plt.subplots(1, 1, figsize=(12, 9))
plot_matches(ax, img1, img2, keypoints1, keypoints2, robust_matches)
plt.axis('off')
plt.title('RANSAC Robust Matches')
plt.show()

plt.imshow(imread('solution_ransac.png'))
plt.axis('off')
plt.title('RANSAC Robust Matches Solution')
plt.show()
```

```
Robust matches shape =  (14, 2)
H =
 [[ 1.01796212e+00  2.42214470e-02  0.00000000e+00]
 [-2.92511861e-02  1.03559397e+00  0.00000000e+00]
 [ 2.03684578e+01  2.58675281e+02  1.00000000e+00]]
```

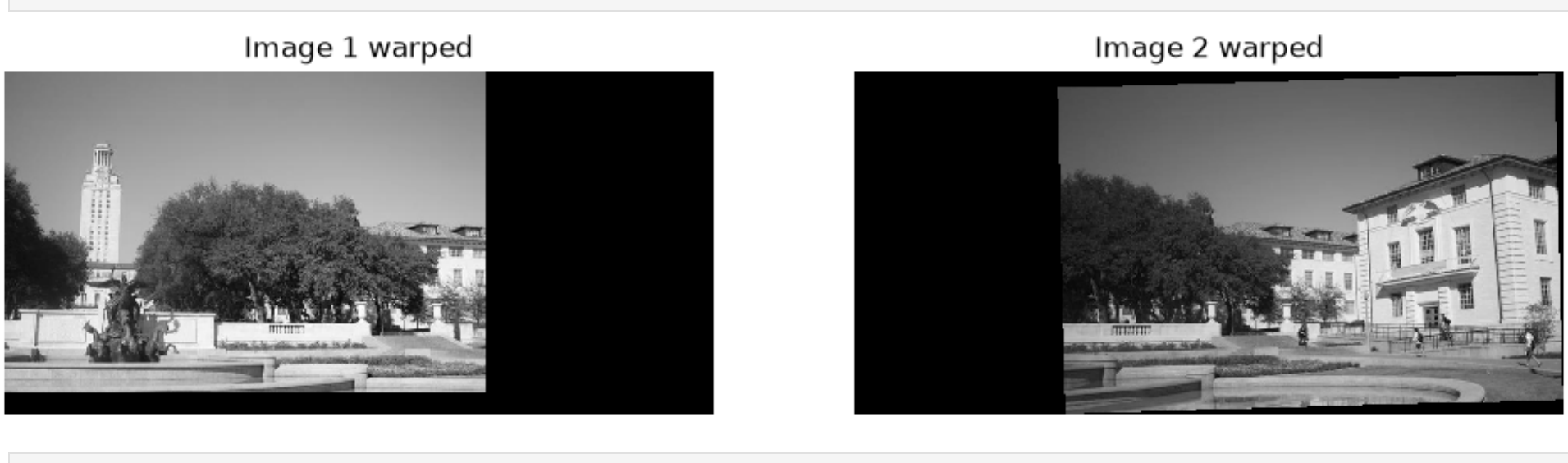




We can now use the tranformation matrix H computed using the robust matches to warp our images and create a better-looking panorama.

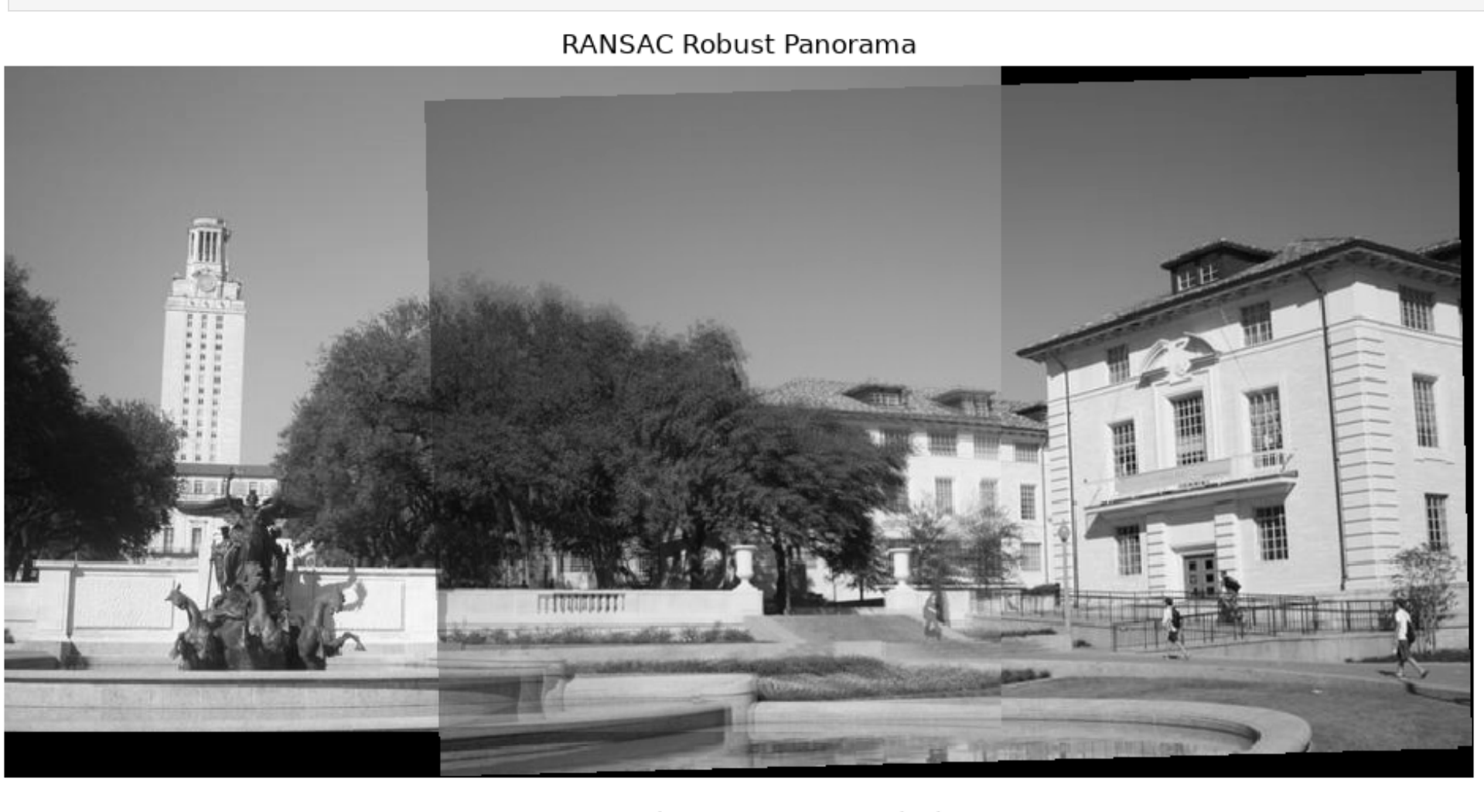
In [75]:

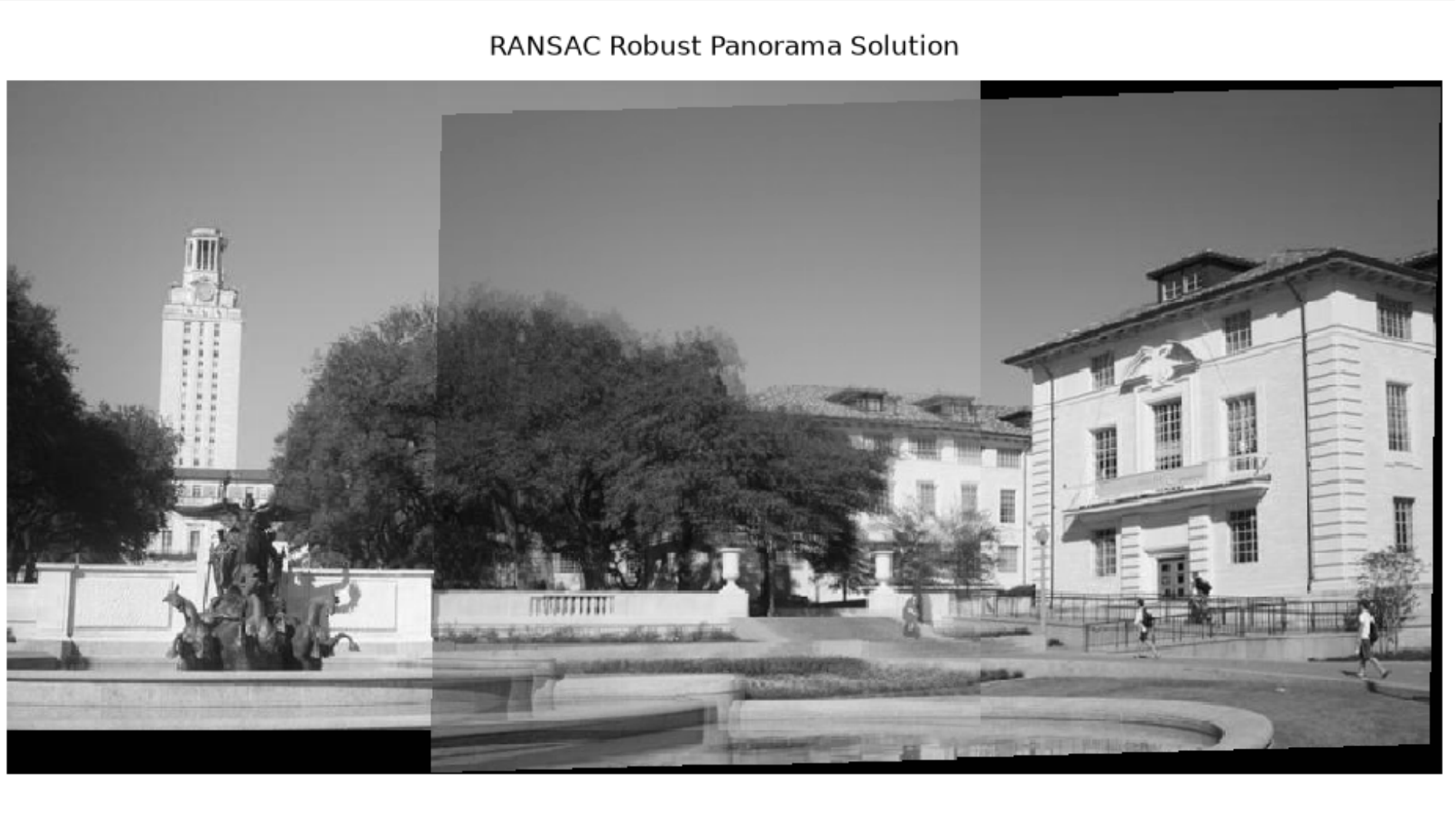
```
output_shape, offset = get_output_space(img1, [img2], [H])

# Warp images into output space
img1_warped = warp_image(img1, np.eye(3), output_shape, offset)
img1_mask = (img1_warped != -1) # Mask == 1 inside the image
img1_warped[~img1_mask] = 0     # Return background values to 0

img2_warped = warp_image(img2, H, output_shape, offset)
img2_mask = (img2_warped != -1) # Mask == 1 inside the image
img2_warped[~img2_mask] = 0     # Return background values to 0

# Plot warped images
plt.subplot(1,2,1)
plt.imshow(img1_warped)
plt.title('Image 1 warped')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(img2_warped)
plt.title('Image 2 warped')
plt.axis('off')

plt.show()
```



In [76]:

```
# Merge the two images
merged = img1_warped + img2_warped

# Track the overlap by adding the masks together
overlap = (img1_mask * 1.0 +  # Multiply by 1.0 for bool -> float conversion
           img2_mask)

# Normalize through division by `overlap` - but ensure the minimum is 1
normalized = merged / np.maximum(overlap, 1)
plt.imshow(normalized)
plt.axis('off')
plt.title('RANSAC Robust Panorama')
plt.show()

plt.imshow(imread('solution_ransac_panorama.png'))
plt.axis('off')
plt.title('RANSAC Robust Panorama Solution')
plt.show()
```





4. Better Image Merging (10 points)

You will notice the blurry region and unpleasant lines in the middle of the final panoramic image. Using a very simple technique called linear blending, we can smooth out a lot of these artifacts.

Currently, all the pixels in the overlapping region are weighted equally. However, since the pixels at the left and right ends of the overlap are very well complemented by the pixels in the other image, they can be made to contribute less to the final panorama.

Linear blending can be done with the following steps:

1. Define left and right margins for blending to occur between
2. Define a weight matrix for image 1 such that:
    - From the left of the output space to the left margin the weight is 1
    - From the left margin to the right margin, the weight linearly decrements from 1 to 0
3. Define a weight matrix for image 2 such that:
    - From the right of the output space to the right margin the weight is 1
    - From the left margin to the right margin, the weight linearly increments from 0 to 1
4. Apply the weight matrices to their corresponding images
5. Combine the images

In `linear_blend` below, implement the linear blending scheme to make the panorama look more natural.

In [77]:

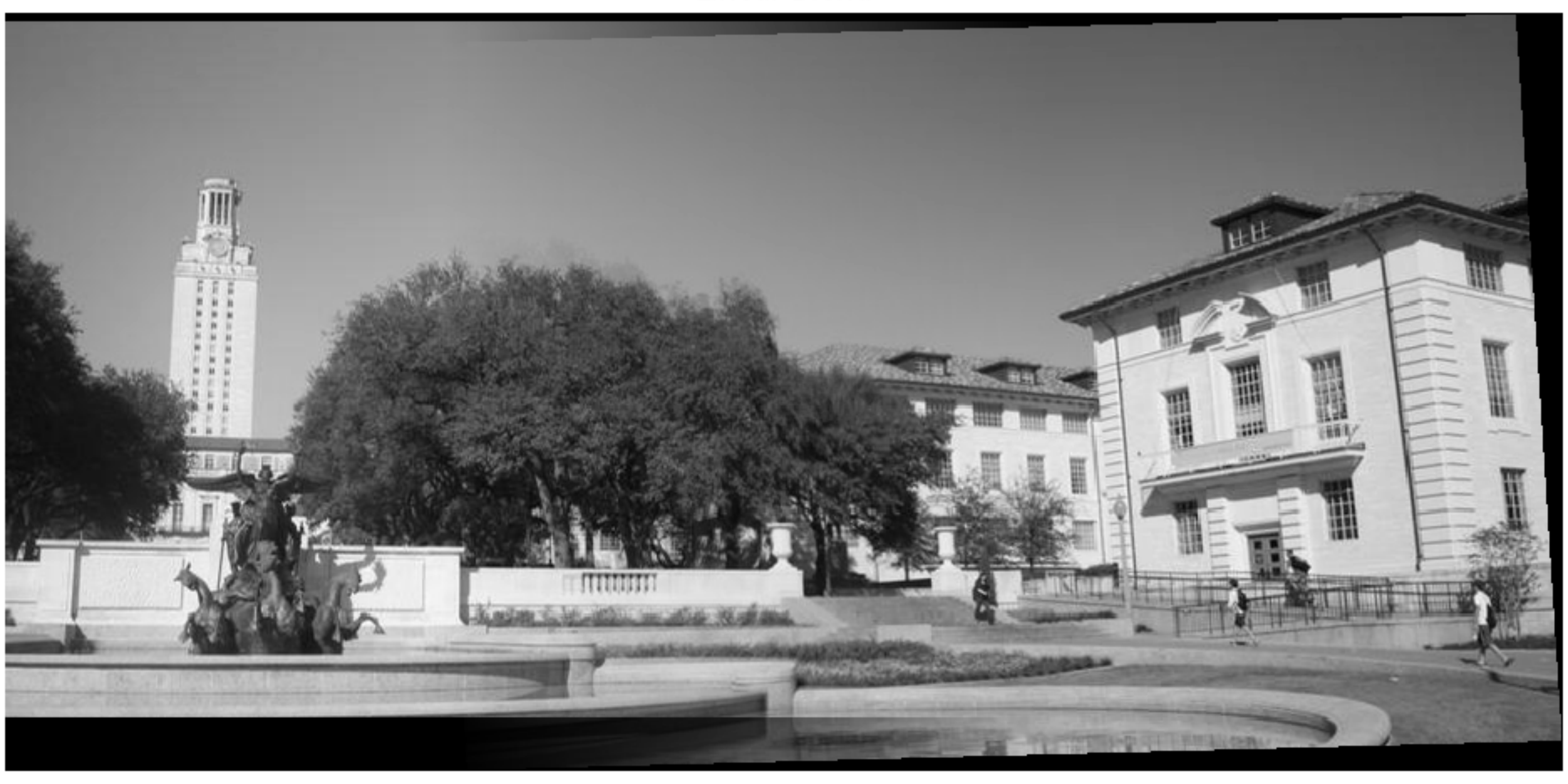
```
def linear_blend(img1_warped, img2_warped):
    """
    Linearly blend img1_warped and img2_warped by following the steps:

    1. Define left and right margins (already done for you)
    2. Define a weight matrices for img1_warped and img2_warped
        np.linspace and np.tile functions will be useful
    3. Apply the weight matrices to their corresponding images
    4. Combine the images

    Args:
        img1_warped: Refernce image warped into output space
        img2_warped: Transformed image warped into output space

    Returns:
        merged: Merged image in output space
    """

    out_H, out_W = img1_warped.shape # Height and width of output space
    img1_mask = (img1_warped != 0)  # Mask == 1 inside the image
    img2_mask = (img2_warped != 0)  # Mask == 1 inside the image

    # Find column of middle row where warped image 1 ends
    # This is where to end weight mask for warped image 1
    right_margin = out_W - np.argmax(np.fliplr(img1_mask)[out_H//2, :].reshape(1, out_W), 1)[0]

    # Find column of middle row where warped image 2 starts
    # This is where to start weight mask for warped image 2
    left_margin = np.argmax(img2_mask[out_H//2, :].reshape(1, out_W), 1)[0]

    ### YOUR CODE HERE
    # Initialize weight maps
    weight1 = np.zeros((out_H, out_W))
    weight2 = np.zeros((out_H, out_W))


    # Blending span
    b_span = right_margin - left_margin

    # 1) For img1_warped:
    #    - left .. left_margin: weight 1
    #    - left_margin .. right_margin: linearly 1 -> 0
    weight1[:, :left_margin] =1
    weight1[:, left_margin: right_margin] = np.tile(np.linspace(1,0, b_span), (out_H,1))



    # 2) For img2_warped:
    #    - right .. right_margin: weight 1 (i.e., right_margin .. end)
    #    - left_margin .. right_margin: linearly 0 -> 1
    weight2[:, left_margin: right_margin] = np.tile(np.linspace(0,1, b_span), (out_H,1))

    weight2[:, right_margin:] = 1

    # Apply masks so only valid pixels contribute
    weight1 *= img1_mask
    weight2 *= img2_mask


    # Blend
    merged = img1_warped*weight1 + img2_warped*weight2

    ### END YOUR CODE

    return merged
```

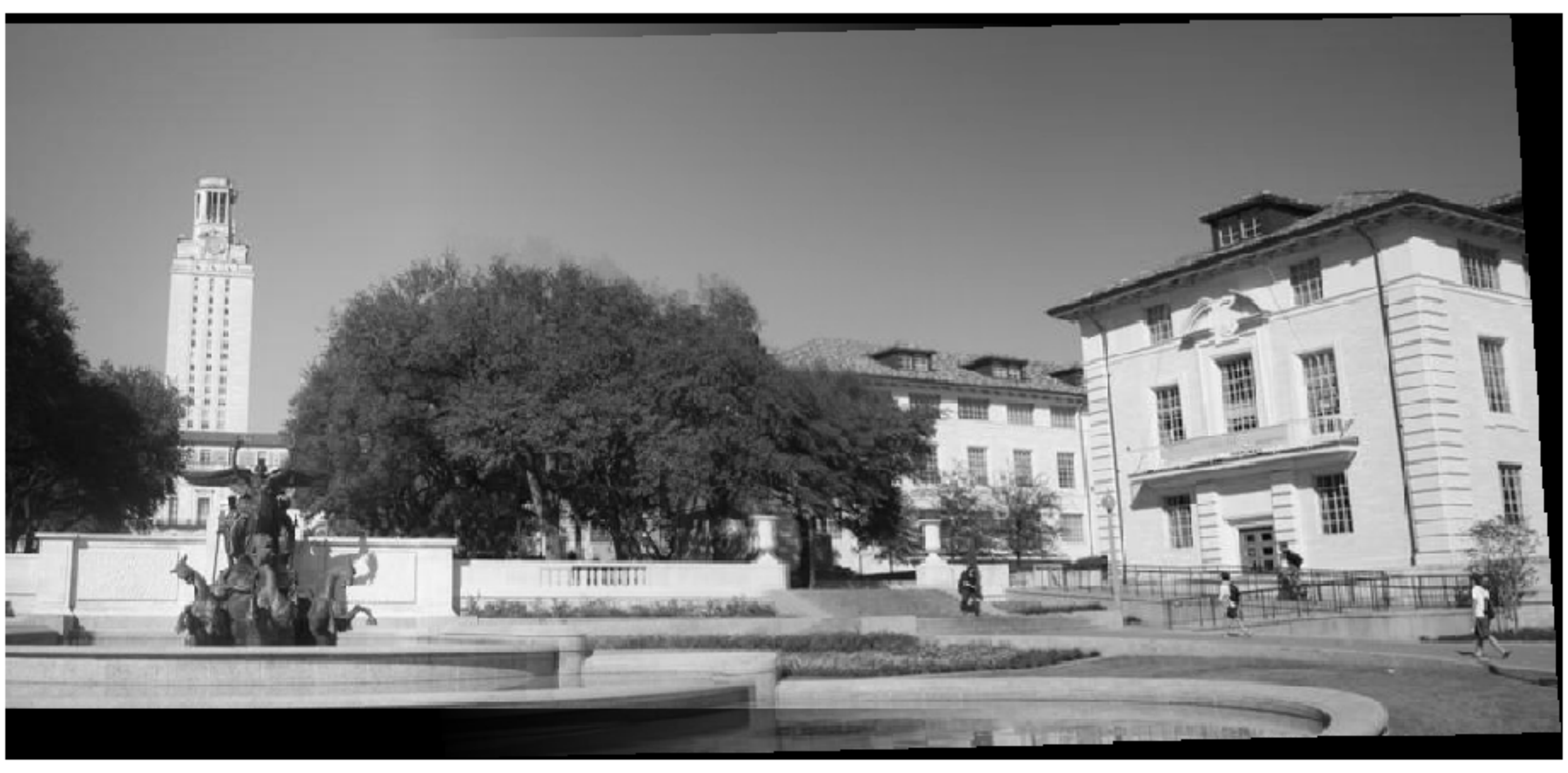
Now let's see how linear blending improves our result.

In [78]:

```
img1 = imread('uttower1.jpg', as_gray=True)
img2 = imread('uttower2.jpg', as_gray=True)

# Set seed to compare output against solution
np.random.seed(131)

# Detect keypoints in both images
ec1_keypoints1 = corner_peaks(harris_corners(img1, window_size=3),
                              threshold_rel=0.05,
                              exclude_border=8)
ec1_keypoints2 = corner_peaks(harris_corners(img2, window_size=3),
                              threshold_rel=0.05,
                              exclude_border=8)

print("EC1 keypoints1 shape = ", ec1_keypoints1.shape)
print("EC1 keypoints2 shape = ", ec1_keypoints2.shape)

# Extract features from the corners
ec1_desc1 = describe_keypoints(img1, ec1_keypoints1,
                           desc_func=simple_descriptor,
                           patch_size=16)
ec1_desc2 = describe_keypoints(img2, ec1_keypoints2,
                           desc_func=simple_descriptor,
                           patch_size=16)

print("EC1 desc1 shape = ", ec1_desc1.shape)
print("EC1 desc2 shape = ", ec1_desc2.shape)

# Match descriptors in image1 to those in image2
ec1_matches = match_descriptors(ec1_desc1, ec1_desc2, 0.7)

H, robust_matches = ransac(ec1_keypoints1, ec1_keypoints2, ec1_matches, threshold=1)
print("Robust matches shape = ", robust_matches.shape)
print("H = \n", H)

output_shape, offset = get_output_space(img1, [img2], [H])
print("Output shape:", output_shape)
print("Offset:", offset)

# Warp images into output space
img1_warped = warp_image(img1, np.eye(3), output_shape, offset)
img1_mask = (img1_warped != -1) # Mask == 1 inside the image
img1_warped[~img1_mask] = 0     # Return background values to 0

img2_warped = warp_image(img2, H, output_shape, offset)
img2_mask = (img2_warped != -1) # Mask == 1 inside the image
img2_warped[~img2_mask] = 0     # Return background values to 0

# Merge the warped images using linear blending scheme
merged = linear_blend(img1_warped, img2_warped)

plt.imshow(merged)
plt.axis('off')
plt.title('Linear Blend')
plt.show()

plt.imshow(imread('solution_linear_blend.png'))
plt.axis('off')
plt.title('Linear Blend Solution')
plt.show()
```

```
EC1 keypoints1 shape =  (396, 2)
EC1 keypoints2 shape =  (627, 2)
EC1 desc1 shape =  (396, 256)
EC1 desc2 shape =  (627, 256)
Robust matches shape =  (43, 2)
H =
 [[ 1.04208721e+00  6.75533965e-02  0.00000000e+00]
 [-2.84886715e-02  1.04672492e+00  0.00000000e+00]
 [ 1.30064490e+01  2.43335682e+02  1.00000000e+00]]
Output shape: [445 915]
Offset: [-4.51408398  0.        ]
```


Linear Blend


Linear Blend Solution